Flight Delay Predictor — Project Write-Up

Distributed Systems for Data Science · New College of Florida · Spring 2026

Prediction Question

Will a US domestic flight arrive more than 15 minutes late, given the airline, route, scheduled departure hour, day of week, month, flight distance, and scheduled duration?

Data Source

Bureau of Transportation Statistics (BTS) 2023 On-Time Performance Dataset — approximately 500,000 domestic flight records covering all major US carriers for the full calendar year 2023. Fields include scheduled and actual departure/arrival times, carrier code, origin/destination airport, distance, and delay cause breakdowns (weather, NAS, carrier, etc.). The dataset is a one-time load for this project; a production system could automate monthly ingestion via the BTS API or a scheduled Lambda trigger pulling new monthly releases from S3.

Pipeline Architecture

```
BTS CSV → S3 (landing) → Databricks PySpark
  Bronze: raw Delta table  (flight_delay_bronze.raw_flights)
  Silver: cleaned + typed  (flight_delay_silver.flights_clean)
  Gold:   feature-engineered (flight_delay_gold.features)
→ MLflow (experiment tracking + model registry)
→ Databricks Model Serving (REST endpoint)
→ Gradio app on Hugging Face Spaces (public URL)
```

Two distributed stages anchor the pipeline: **AWS S3** as the cloud object store landing zone, and **Databricks PySpark** for all medallion-layer transformations. Delta Lake persists every layer. The Databricks Model Serving endpoint handles inference requests. The Gradio frontend is hosted free on Hugging Face Spaces.

Model Approach

A **Random Forest Classifier** (200 trees, max depth 10, class-weight balanced) was trained on 11 engineered features: month, day of week, departure hour, weekend flag, distance, scheduled duration, route-level historical delay rate, airline-level historical delay rate, and encoded airline/origin/destination identifiers. A Gradient Boosting challenger was also trained for comparison. Both runs are tracked in MLflow under `/Shared/flight-delay-predictor`; the best model (by ROC-AUC) was promoted to the Production stage of the MLflow Model Registry and deployed to a Databricks Model Serving endpoint.

Metric	Value
ROC-AUC	0.78
F1 Score	0.71
Accuracy	0.74

What I Learned

The hardest part was not the machine learning — it was the **plumbing between services**. Getting Databricks Model Serving to accept the exact JSON schema the Gradio frontend sends required more iteration than expected, particularly around encoding categorical variables consistently between training and inference. If I rebuilt this, I would package the label encoders as part of the MLflow model artifact (using a custom `pyfunc` wrapper) rather than managing them separately in S3 — that would eliminate the most fragile part of the serving pipeline. I also underestimated how useful MLflow's experiment comparison UI is; having every hyperparameter and metric logged made picking the production model genuinely easy.